ITHU NAMMA KADA

Cryptographic Licensing & Offline Activation Manual

ITHU NAMMA KADA: Offline Licensing & Activation System Architecture

This architecture blueprint details the design, security, and implementation of a professional, offline-first licensing and activation system for **ITHU NAMMA KADA** (commercial offline billing software built with React, Node.js, Electron, Express, and SQLite).

$0

1. System Architecture & Module Responsibilities

The software is structured as a multi-module repository. Each module has a specific responsibility to separate concerns, facilitate local execution without internet, and enable administrative oversight.

Project Layout

```
text
ITHU-NAMMA-KADA/
├── frontend/               # React UI for the Desktop App (Billing, Reports, Adm
├── backend/                # Local Node.js Express service running on the user's
├── electron/               # Electron Main Process, Preload, and OS-level packag
├── licensing-server/       # Centralized online API server (Node.js/Express + Mo
├── admin-portal/           # React dashboard for managing licenses and customers
└── shared/                 # Shared TypeScript models, utility helpers, and DTO
```

Module Responsibilities

$0

$0

$0

$0

$0

$0

$0

$0

System Integration Flow

```
mermaid
graph TD
    subgraph Local Client Machine (Offline Billing App)
        FE[React Frontend] <-->|HTTP/IPC| BE[Local Express Backend]
        BE <-->|Prisma / SQLCipher| DB[(Local SQLite DB)]
        EL[Electron Main Process] -->|Launch / Secure Check| BE
        EL -->|Read / Verify| LIC[(Signed Local License File)]
        BE -->|Print Raw TSPL / ESC-POS| PR[Printers USB/COM]
    end

    subgraph Cloud Server (Online Web Services)
        AP[Admin Web Portal] <-->|HTTPS| LS[Licensing Server]
        LS <-->|MongoDB Driver| MDB[(Cloud MongoDB)]
    end

    BE -.->|Activation Request - HTTPS| LS
    AP -.->|Offline Activation .lic file| LIC
```

$0

2. Cryptographic Licensing (RSA vs Ed25519)

We will use **Ed25519** (Edwards-curve Digital Signature Algorithm) instead of **RSA** for license signing and verification.

Comparison Analysis

$0

$0

$0

$0

$0

$0

$0

Ed25519 Cryptographic Code Snippets

1. Generate Key Pair (Licensing Server - One Time Setup)

Save these keys securely. The public key will be bundled with the Electron application, while the private key remains highly secure on the Licensing Server.

```
javascript
// licensing-server/scripts/generateKeys.js
const crypto = require('crypto');
const fs = require('fs');

const { publicKey, privateKey } = crypto.generateKeyPairSync('ed25519', {
    publicKeyEncoding: { type: 'spki', format: 'pem' },
    privateKeyEncoding: { type: 'pkcs8', format: 'pem' }
});

fs.writeFileSync('public.key', publicKey);
fs.writeFileSync('private.key', privateKey);
console.log('Ed25519 keypair successfully generated.');
```

2. Digitally Sign License Payload (Licensing Server)

```
javascript
// licensing-server/services/signer.service.js
const crypto = require('crypto');
const fs = require('fs');

const privateKey = fs.readFileSync('private.key', 'utf8');

function signLicense(licenseData) {
    // 1. Serialize the license data into a canonical string format
    const serializedPayload = JSON.stringify(licenseData);

    // 2. Create the digital signature
    const signature = crypto.sign(
        null, // Ed25519 does not require a hash algorithm parameter in Node.js
        Buffer.from(serializedPayload),
        privateKey
    );

    // 3. Return signature in hex format along with raw payload data
    return {
        data: licenseData,
        signature: signature.toString('hex')
    };
}
```

3. Verify Signature in Electron Desktop App

```
javascript
// electron/src/licenseVerifier.js
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');

// Public key is bundled locally in the Electron build (read-only assets)
const publicKeyPath = path.join(__dirname, 'assets', 'public.key');
const publicKey = fs.readFileSync(publicKeyPath, 'utf8');

function verifyLicenseFile(licenseFilePath, machineId) {
    try {
        if (!fs.existsSync(licenseFilePath)) {
            return { valid: false, error: 'License file not found.' };
        }

        const rawContent = fs.readFileSync(licenseFilePath, 'utf8');
        const { data, signature } = JSON.parse(rawContent);

        // Step A: Verify cryptographic integrity of the signature
        const serializedPayload = JSON.stringify(data);
        const isSignatureValid = crypto.verify(
            null,
            Buffer.from(serializedPayload),
            publicKey,
            Buffer.from(signature, 'hex')
        );

        if (!isSignatureValid) {
            return { valid: false, error: 'License signature is tampered!' };
        }

        // Step B: Verify Machine ID matching
        if (data.machineId !== machineId) {
            return { valid: false, error: 'License bound to a different machine.'
        }

        // Step C: Verify Expiration Date
        const currentTimestamp = Date.now();
        if (data.expiresAt && currentTimestamp > new Date(data.expiresAt).getTime
            return { valid: false, error: 'License has expired.', data };
```

```
        }

        return { valid: true, data };
    } catch (err) {
        return { valid: false, error: `Verification failed: ${err.message}` };
    }
}
```

$0

3. Reliable Machine ID (Hardware Fingerprint) Generation

To tie the software to a single PC, we must generate a reliable hardware fingerprint. Relying only on MAC addresses is dangerous (they change when toggling WiFi/VPN adapters or docks). Relying on BIOS UUID might return generic values like `"To Be Filled By O.E.M."` or `"00000000-0000-0000-0000-000000000000"`.

Combined Weighted Fingerprint

We generate a composite ID by extracting and hashing multiple hardware properties: 1. **Motherboard Serial Number** (Highly stable) 2. **CPU Serial/Processor ID** (Failsafe identifier) 3. **OS System UUID** (Tied to the Windows installation)

Implementation Code

```
javascript
// electron/src/machineId.js
const { execSync } = require('child_process');
const crypto = require('crypto');

function getMotherboardSerial() {
    try {
        const stdout = execSync('wmic baseboard get serialnumber').toString();
        const lines = stdout.split('\n');
        const serial = lines[1] ? lines[1].trim() : '';
        return serial && !isGenericPlaceholder(serial) ? serial : '';
    } catch {
        return '';
    }
}

function getCpuId() {
    try {
        const stdout = execSync('wmic cpu get processorid').toString();
        const lines = stdout.split('\n');
        const cpuId = lines[1] ? lines[1].trim() : '';
        return cpuId && !isGenericPlaceholder(cpuId) ? cpuId : '';
    } catch {
        return '';
    }
}

function getSystemUuid() {
    try {
        const stdout = execSync('wmic path win32_computersystemproduct get uuid')
        const lines = stdout.split('\n');
        const uuid = lines[1] ? lines[1].trim() : '';
        return uuid && !isGenericPlaceholder(uuid) ? uuid : '';
    } catch {
        return '';
    }
}

function isGenericPlaceholder(val) {
    const v = val.toLowerCase().replace(/[^a-z0-9]/g, '');
    const genericTerms = [
```

```
        'tobefilledbyoem', 'none', 'default', 'unknown', 'notapplicable',
        '00000000', '11111111', 'fillbyoem'
    ];
    return genericTerms.some(term => v.includes(term)) || v.length < 4;
}

function generateStableMachineId() {
    const motherboard = getMotherboardSerial();
    const cpu = getCpuId();
    const uuid = getSystemUuid();

    // Fallback if hardware APIs are blocked (e.g. standard user accounts without
    const fallbackString = `${process.arch}-${process.platform}-${process.env.COM

    // Combine component strings. We sort them to preserve deterministic behavior
    const hardwareTokens = [motherboard, cpu, uuid].filter(Boolean);

    let baseString = '';
    if (hardwareTokens.length >= 2) {
        baseString = hardwareTokens.join('|');
    } else {
        // Use fallback if at least two identifiers couldn't be extracted
        baseString = `${hardwareTokens.join('|')}|${fallbackString}`;
    }

    return crypto.createHash('sha256').update(baseString).digest('hex').toUpperCa
}

module.exports = { generateStableMachineId };
```

$0

4. Activation Workflows

Online Activation Workflow

```
mermaid
sequenceDiagram
    autonumber
    actor Customer
    participant Desktop as Electron Desktop App
    participant Server as Licensing Server (Cloud)
    participant Database as MongoDB (Cloud)

    Customer->>Desktop: Launches app first time. Enters License Key & Shop Name
    Note over Desktop: App computes local Machine ID via hardware hashes
    Desktop->>Server: POST /license/activate (LicenseKey, MachineID, ShopName, Ve
    Server->>Database: Queries License table by Key

    alt License Key Invalid or Expired
        Server-->>Desktop: 400 Bad Request (Error: Invalid Key)
        Desktop-->>Customer: Display error: "License key is invalid or expired"
    else Activation Limit Reached (Different Machine ID)
        Server-->>Desktop: 403 Forbidden (Error: Activation limit exceeded)
        Desktop-->>Customer: Display: "License is already active on another machi
    else Activation Allowed
        Server->>Database: Inserts/Updates Activation record
        Server->>Server: Generates signed license structure (Ed25519 Private Key)
        Server-->>Desktop: 200 OK (Signed JSON string payload)
        Note over Desktop: Desktop verifies cryptographic signature using bundled
        Desktop->>Desktop: Writes signed payload to local license.lic file
        Desktop-->>Customer: Displays: "Activation Successful! Software Unlocked.
    end
```

Offline Activation Workflow (For remote locations without internet)

This process allows activation via manual transfer of a signed license file.

```
mermaid
sequenceDiagram
    autonumber
    actor Customer
    participant Desktop as Offline Electron App
    actor Admin as Admin Team / WhatsApp Support
    participant Portal as Admin Web Portal
    participant Server as Licensing Server (Cloud)

    Customer->>Desktop: Launches app. Selects "Offline Activation"
    Note over Desktop: Desktop generates and shows the Machine ID
    Customer->>Admin: Sends Machine ID + License Key via WhatsApp
    Admin->>Portal: Enters License Key & Machine ID
    Portal->>Server: POST /license/offline-issue (LicenseKey, MachineID, AdminTok
    Note over Server: Validates key status, signs license payload bound to the cu
    Server-->>Portal: Returns signed license JSON structure
    Portal->>Admin: Download license as file: ithu_namma_kada.lic
    Admin->>Customer: Sends ithu_namma_kada.lic via WhatsApp
    Customer->>Desktop: Clicks "Import License File" and uploads .lic file
    Note over Desktop: App verifies signature and validates that license machineI
    Desktop->>Desktop: Saves verified file to ProgramData location
    Desktop-->>Customer: Software Activates Successfully
```

$0

5. Machine Transfer & Reset Workflows

To prevent piracy, a license must restrict simultaneous active machines (typically Max Activations = 1). When a customer changes computers or their hardware fails, we use a two-path workflow:

Path A: Graceful Deactivation (Self-Service)

The customer has access to their activated machine and wishes to migrate software to a new machine.

1. **Trigger**: In the desktop app, navigate to **Settings > License Management > Deactivate License**. 2. **Online Call**: App calls `POST /license/deactivate` with `licenseKey` and the current `machineId`. 3. **Server Action**: Licensing server removes the Activation record from the database. 4. **Local Action**:

$0

$0

$0

Path B: Admin-Led Reset (Hardware Crash / Windows Reinstall)

The customer's hard drive crashed, or they reinstalled Windows and their Machine ID changed, making graceful deactivation impossible.

1. **Trigger**: Customer contacts Support/WhatsApp with their License Key. 2. **Verification**: Support agent searches the License Key on the **Admin Portal**. 3. **Action**: Support agent reviews activation logs (checks matching customer details) and clicks **Reset Machine ID / Decouple Device**. 4. **Server Database Action**:

$0

$0

5. **Re-activation**: The user launches the app on the new/rebuilt machine, inputs their License Key, and registers successfully online.

$0

6. Feature-Based Licensing

The system controls access to high-value features (e.g. barcode generation or cloud backup) based on the subscription tier in the signed license file.

Structure of the Signed License File (`license.lic`)

```
json
{
  "data": {
    "licenseKey": "INK-PRO-9831-2947-8102",
    "shopName": "Murugan Stores",
    "ownerMobile": "+919876543210",
    "machineId": "D59B25A7C4B06DE955E1A18671FDF69096238AA1CE99F55BE7B060938AA4FF6
    "planType": "Annual",
    "issuedAt": "2026-07-22T17:00:00Z",
    "expiresAt": "2027-07-22T17:00:00Z",
    "features": {
      "billing": true,
      "inventory": true,
      "barcode_printing": true,
      "thermal_printing": true,
      "whatsapp_invoice": true,
      "daily_sales_report": true,
      "gst_reports": true,
      "multiple_users": false,
      "cloud_backup": false
    }
  },
  "signature": "e605d3b6f2b1d7d...c4293fef"
}
```

Implementing Frontend Feature Guards (React Context)

```tsx
// frontend/src/context/LicenseContext.tsx
import React, { createContext, useContext, useState, useEffect } from 'react';

interface LicenseFeatures {
  billing: boolean;
  inventory: boolean;
  barcode_printing: boolean;
  thermal_printing: boolean;
  whatsapp_invoice: boolean;
  daily_sales_report: boolean;
  gst_reports: boolean;
  multiple_users: boolean;
  cloud_backup: boolean;
}

interface LicenseContextType {
  isActivated: boolean;
  features: LicenseFeatures | null;
  daysRemaining: number;
  hasFeature: (feature: keyof LicenseFeatures) => boolean;
  loading: boolean;
}

const LicenseContext = createContext<LicenseContextType | null>(null);

export const LicenseProvider: React.FC<{ children: React.ReactNode }> = ({ childr
  const [isActivated, setIsActivated] = useState(false);
  const [features, setFeatures] = useState<LicenseFeatures | null>(null);
  const [daysRemaining, setDaysRemaining] = useState(0);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Call Electron IPC to read active license
    (window as any).api.send('check-license-status');
    (window as any).api.receive('license-status-response', (event: any, arg: any)
      if (arg.valid) {
        setIsActivated(true);
        setFeatures(arg.data.features);

        const diffTime = new Date(arg.data.expiresAt).getTime() - Date.now();
```

```
        const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
        setDaysRemaining(diffDays);
      } else {
        setIsActivated(false);
        setFeatures(null);
      }
      setLoading(false);
    });
  }, []);

  const hasFeature = (feature: keyof LicenseFeatures): boolean => {
    if (!isActivated || !features) return false;
    return !!features[feature];
  };

  return (
    <LicenseContext.Provider value={{ isActivated, features, daysRemaining, hasFe
      {children}
    </LicenseContext.Provider>
  );
};

export const useLicense = () => {
  const context = useContext(LicenseContext);
  if (!context) throw new Error('useLicense must be used within a LicenseProvider
  return context;
};
```

$0

7. License Expiry & Warning System

To support different subscription periods—specifically the **3-Month Plan (Quarterly)**—the Licensing Server sets the `expiresAt` field in the signed JSON payload to exactly **90 days** from the date of activation/issuance.

Expiry Notifications

$0

$0

$0

How to Verify if Expiry Logic is Working

You can verify that the 3-month restriction is working by checking four distinct checkpoints:

Checkpoint 1: Inspect the Local Decrypted License File (`license.lic`)

Open the generated license file at `%PROGRAMDATA%\IthuNammaKada\license.lic`. Verify the structure of the JSON:

$0

$0

$0

Checkpoint 2: Mock Expiry Dates for Fast Testing

Since waiting 90 days to test is impractical, you can modify the date on a trial license to test the triggers: 1. **Test the 30-Day Warning Banner**: Create or sign a test license file with `expiresAt` set to **25 days in the future**. Launch the app. A yellow banner should appear at the top: *"Your license expires in 25 days. Click here to renew."* 2. **Test the 7-Day Warning Modal**: Create a test license file with `expiresAt` set to **5 days in the future**. Launch the app. A pop-up dialog should interrupt the dashboard: *"Warning: Your license will expire in 5 days. Please contact support to avoid service interruption."* 3. **Test the Hard Lockout Screen**: Create a test license file with `expiresAt` set to **10 minutes in the past**. Launch the app. The app must immediately redirect to the Activation/Lock Screen, disabling all POS checkout buttons, database writes, and printing ports.

Checkpoint 3: Test System Clock Rolled Forward

To verify that the client checks system limits correctly: 1. Activate a valid 3-month license. 2. Close the application. 3. Manually change your Windows OS system date to **4 months in the future** (bypassing the 3-month expiry window). 4. Launch the application. 5. The verifier will read the system clock, check it against `expiresAt`, detect that it has exceeded the expiry date, and display the **License Expired** screen.

Checkpoint 4: Test Clock Tampering Guard (Rolling Clock Back)

To verify that users cannot roll back their Windows clock to cheat the 3-month limit: 1. Open the app on a valid 3-month license and write a transaction (e.g. create a sales bill) at the current date/time. 2. Close the app. 3. Change your Windows OS system date **backward by 1 year**. 4. Launch the app. 5. The security system will run `checkClockTampering()`. It detects that the system clock (`2025`) is earlier than the latest transaction in the database (`2026`) or the last startup log file. 6. The app will

immediately throw a **"System Clock Tampering Detected"** error and lock the software, preventing bypass.

Clock Tampering Prevention Code

```
javascript
// backend/src/services/clockGuard.service.js
const fs = require('fs');
const path = require('path');
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function checkClockTampering() {
    const systemTime = new Date();

    // 1. Check against the last recorded transaction/sales invoice in SQLite
    const latestBill = await prisma.salesBill.findFirst({
        orderBy: { invDate: 'desc' }
    });

    if (latestBill && latestBill.invDate > systemTime) {
        return { tampered: true, error: 'System clock has been rolled back. Local
    }

    // 2. Check against the last startup timestamp stored in local configurations
    const configPath = path.join(process.env.APPDATA, 'IthuNammaKada', 'app.cfg')
    if (fs.existsSync(configPath)) {
        const configData = JSON.parse(fs.readFileSync(configPath, 'utf8'));
        const lastRunTime = new Date(configData.lastRunTime);

        if (lastRunTime > systemTime) {
            return { tampered: true, error: 'Clock rollback detected (previous sy
        }

        // Update last run time to current time
        configData.lastRunTime = systemTime.toISOString();
        fs.writeFileSync(configPath, JSON.stringify(configData, null, 2));
    } else {
        // Initial setup of configuration file
        const configDir = path.dirname(configPath);
        if (!fs.existsSync(configDir)) fs.mkdirSync(configDir, { recursive: true
        fs.writeFileSync(configPath, JSON.stringify({ lastRunTime: systemTime.toI
    }

    return { tampered: false };
}
```

```
module.exports = { checkClockTampering };
```

$0

8. Admin Web Portal Design

The portal is a web-based responsive dashboard (React + Tailwind CSS) used to manage customer lifecycle events.

Wireframe Mockup Layout

```
text
+-------------------------------------------------------------------------------
|  [Logo] ITHU NAMMA KADA ADMIN                  Welcome, Admin | [Logout]
+-------------------------------------------------------------------------------
|  [Sidebar]             |  [Dashboard Metrics]
|  * Dashboard           |  Total Licenses: 1,420    Active: 1,110    Suspended:
|  * Licenses           +-----------------------------------------------------
|  * Customers           |  [License Manager]                        [+ Create Ke
|  * Activations         |  Search Key/Shop Name: [________________] [Filter: All
|  * Audit Logs          |
|  * System Settings     |  +--------------------------------------------------
|                        |  | Key | Shop Name | Tier | Expiry | Active Machine
|                        |  |--------------------------------------------------
|                        |  | INK-PRO.. | Murugan S.  | Annual | 22-07-27 | D59B2
|                        |  | [Edit] [Renew] [Suspend] [Reset Device] [Download]
|                        |  +--------------------------------------------------
+-------------------------------------------------------------------------------
|  [Action Panel: Create New License]
|  Shop Name: [_________] Mobile: [________] Tier: (Lifetime / Annual / Monthly)
|  Features: [x] Billing  [x] Inventory  [x] Barcode  [ ] Cloud Sync  [x] WhatsAp
|  [Generate License Key]
+-------------------------------------------------------------------------------
```

$0

9. Database Design

MongoDB Schema (Licensing Server - Cloud Database)

```javascript
// licensing-server/models/Schemas.js
const mongoose = require('mongoose');

// 1. Customer Schema
const CustomerSchema = new mongoose.Schema({
    shopName: { type: String, required: true, trim: true },
    contactName: { type: String, required: true },
    mobileNo: { type: String, required: true, unique: true },
    email: { type: String, trim: true },
    gstNo: { type: String, trim: true },
    address: String,
    createdAt: { type: Date, default: Date.now }
});

// 2. License Schema
const LicenseSchema = new mongoose.Schema({
    licenseKey: { type: String, required: true, unique: true },
    customerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Customer', required
    planType: { type: String, enum: ['Lifetime', 'Annual', 'Monthly', 'Trial'], r
    status: { type: String, enum: ['Active', 'Suspended', 'Expired'], default: 'A
    features: {
        billing: { type: Boolean, default: true },
        inventory: { type: Boolean, default: true },
        barcode_printing: { type: Boolean, default: false },
        thermal_printing: { type: Boolean, default: true },
        whatsapp_invoice: { type: Boolean, default: false },
        daily_sales_report: { type: Boolean, default: true },
        gst_reports: { type: Boolean, default: false },
        multiple_users: { type: Boolean, default: false },
        cloud_backup: { type: Boolean, default: false }
    },
    maxActivations: { type: Number, default: 1 },
    expiresAt: { type: Date },
    createdAt: { type: Date, default: Date.now }
});

// 3. Activation Schema (Binds License with physical Machine IDs)
const ActivationSchema = new mongoose.Schema({
    licenseId: { type: mongoose.Schema.Types.ObjectId, ref: 'License', required:
    machineId: { type: String, required: true },
    softwareVersion: { type: String, required: true },
```

```
    ipAddress: String,
    osDetails: String,
    activatedAt: { type: Date, default: Date.now },
    status: { type: String, enum: ['Active', 'Deactivated'], default: 'Active' }
});

// 4. Audit Log Schema
const AuditLogSchema = new mongoose.Schema({
    action: { type: String, required: true }, // e.g. "LICENSE_CREATED", "MACHINE
    performedBy: { type: String, required: true }, // e.g. "admin_username" or "s
    details: mongoose.Schema.Types.Mixed,
    timestamp: { type: Date, default: Date.now }
});

module.exports = {
    Customer: mongoose.model('Customer', CustomerSchema),
    License: mongoose.model('License', LicenseSchema),
    Activation: mongoose.model('Activation', ActivationSchema),
    AuditLog: mongoose.model('AuditLog', AuditLogSchema)
};
```

PostgreSQL Schema (Relational Alternative - Cloud Database)

Use this schema if you prefer PostgreSQL for relational modeling.

```sql
-- DDL for PostgreSQL Database

CREATE TYPE plan_enum AS ENUM ('Lifetime', 'Annual', 'Monthly', 'Trial');
CREATE TYPE license_status AS ENUM ('Active', 'Suspended', 'Expired');
CREATE TYPE activation_status AS ENUM ('Active', 'Deactivated');

CREATE TABLE customers (
    id SERIAL PRIMARY KEY,
    shop_name VARCHAR(255) NOT NULL,
    contact_name VARCHAR(255) NOT NULL,
    mobile_no VARCHAR(20) UNIQUE NOT NULL,
    email VARCHAR(255),
    gst_no VARCHAR(15),
    address TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE licenses (
    id SERIAL PRIMARY KEY,
    license_key VARCHAR(100) UNIQUE NOT NULL,
    customer_id INTEGER REFERENCES customers(id) ON DELETE CASCADE,
    plan_type plan_enum NOT NULL,
    status license_status DEFAULT 'Active',
    max_activations INTEGER DEFAULT 1,
    expires_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE license_features (
    id SERIAL PRIMARY KEY,
    license_id INTEGER UNIQUE REFERENCES licenses(id) ON DELETE CASCADE,
    billing BOOLEAN DEFAULT TRUE,
    inventory BOOLEAN DEFAULT TRUE,
    barcode_printing BOOLEAN DEFAULT FALSE,
    thermal_printing BOOLEAN DEFAULT TRUE,
    whatsapp_invoice BOOLEAN DEFAULT FALSE,
    daily_sales_report BOOLEAN DEFAULT TRUE,
    gst_reports BOOLEAN DEFAULT FALSE,
    multiple_users BOOLEAN DEFAULT FALSE,
    cloud_backup BOOLEAN DEFAULT FALSE
);
```

```
CREATE TABLE activations (
    id SERIAL PRIMARY KEY,
    license_id INTEGER REFERENCES licenses(id) ON DELETE CASCADE,
    machine_id VARCHAR(255) NOT NULL,
    software_version VARCHAR(20) NOT NULL,
    ip_address VARCHAR(45),
    os_details VARCHAR(255),
    activated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status activation_status DEFAULT 'Active'
);

CREATE TABLE audit_logs (
    id SERIAL PRIMARY KEY,
    action VARCHAR(100) NOT NULL,
    performed_by VARCHAR(100) NOT NULL,
    details TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Index mappings to optimize queries
CREATE INDEX idx_license_key ON licenses(license_key);
CREATE INDEX idx_activation_machine ON activations(machine_id);
```

$0

10. Security & Anti-Tampering Matrix

$0

$0

$0

$0

$0

$0

$0

$0

11. API Specifications

All endpoints use JSON payloads and require an `X-Api-Key` or JSON Web Token (JWT) authorization header for administration tasks.

1. `POST /license/activate`

Activates software online during initial client boot.

$0

$0

```
json
{
  "licenseKey": "INK-PRO-9831-2947-8102",
  "machineId": "D59B25A7C4B06DE955E1A18671FDF69096238AA1CE99F55BE7B060938AA4FF6A"
  "shopName": "Murugan Stores",
  "softwareVersion": "1.0.0",
  "osDetails": "Windows 11 Home v22H2"
}
```

$0

```
json
{
  "success": true,
  "message": "Activation successful.",
  "licenseFile": {
    "data": {
      "licenseKey": "INK-PRO-9831-2947-8102",
      "shopName": "Murugan Stores",
      "machineId": "D59B25A7C4B06DE955E1A18671FDF69096238AA1CE99F55BE7B060938AA4F
      "planType": "Annual",
      "issuedAt": "2026-07-22T17:20:00Z",
      "expiresAt": "2027-07-22T17:20:00Z",
      "features": {
        "billing": true,
        "inventory": true,
        "barcode_printing": true,
        "thermal_printing": true,
        "whatsapp_invoice": true,
        "daily_sales_report": true,
        "gst_reports": true,
        "multiple_users": false,
        "cloud_backup": false
      }
    },
    "signature": "3045022100e405a306bd7835b0d0c362ba0a2491a92df4b1bb24f33190df"
  }
}
```

$0

```
json
{
  "success": false,
  "errorCode": "ACTIVATION_LIMIT_EXCEEDED",
  "message": "License key is active on another machine. Perform machine deactivat
}
```

2. `POST /license/offline`

Issues an offline `.lic` activation payload from the licensing server. Called via the Admin Portal.

$0

```
json
{
  "licenseKey": "INK-PRO-9831-2947-8102",
  "machineId": "D59B25A7C4B06DE955E1A18671FDF69096238AA1CE99F55BE7B060938AA4FF6A"
}
```

$0

```
json
{
  "success": true,
  "downloadUrl": "https://api.ithunammakada.in/licenses/download/INK-PRO-9831-294
  "licenseFile": {
    "data": { ... },
    "signature": "..."
  }
}
```

3. `POST /license/deactivate`

Deactivates a license from a local machine, freeing the activation slot.

$0

```
json
{
  "licenseKey": "INK-PRO-9831-2947-8102",
  "machineId": "D59B25A7C4B06DE955E1A18671FDF69096238AA1CE99F55BE7B060938AA4FF6A"
}
```

$0

```
json
{
  "success": true,
  "message": "Device deactivated successfully. The license slot is now free."
}
```

4. `POST /license/reset-machine`

Admin command to reset activation without deactivating from client. Called via Admin Portal.

$0

```
json
{
  "licenseKey": "INK-PRO-9831-2947-8102",
  "reason": "Hard Drive crashed, system rebuilt"
}
```

$0

```
json
{
  "success": true,
  "message": "All active machine associations for this key have been cleared."
}
```

$0

12. Local Storage Strategy on Windows

The software uses directories that are accessible to standard Windows users but are resilient to data clearing.

Windows Folder Structure

$0

$0

$0

$0

$0

$0

Database Encryption: SQLite + SQLCipher

To prevent customers from opening the SQLite database using DB Browser and modifying pricing or sales data, we encrypt the database file using **SQLCipher**.

```
javascript
// backend/src/database.js
const sqlite3 = require('sqlite3');
const { open } = require('sqlite');
const { generateStableMachineId } = require('./machineId');

async function initializeDatabase() {
    const dbPath = `${process.env.APPDATA}/IthuNammaKada/db/pos.db`;

    const db = await open({
        filename: dbPath,
        driver: sqlite3.Database
    });

    // Generate db passphrase using Machine ID + custom pepper
    const rawPassphrase = generateStableMachineId();
    const secureKey = crypto.createHmac('sha256', 'KADA-PEPPER-9872').update(rawP

    // Apply SQLCipher passphrase execution command before querying
    await db.run(`PRAGMA key = '${secureKey}'`);

    return db;
}
```

$0

13. Backup & Recovery Design

```
mermaid
graph LR
    DB[(SQLite DB)] -->|Automatic Daily / On Close| BK[Backup Engine]
    BK -->|AES-256-GCM Encrypt| LDIR[(Local Document Backup Directory)]
    BK -.->|Optional Cloud Sync| CLD[(Cloud Server Storage)]
```

Disaster Recovery: Physically Destroyed Hardware (Burnt/Stolen PC)

If the client's computer is completely destroyed (e.g. fire, power surge, theft) and the local storage drive cannot be salvaged, follow this recovery process:

Phase 1: License Migration (Admin Portal)

Since the user cannot deactivate the license from the old machine, the support team handles this: 1. The client contacts support with their **Shop Name** or **License Key**. 2. The support admin logs into the central **Admin Portal** and navigates to the customer profile. 3. The admin reviews the activation logs and clicks **Reset Machine ID** (executing a `POST /license/reset-machine` request to the Licensing Server). 4. The server deletes the old hardware fingerprint bound to the key, freeing it up for registration on a new computer.

Phase 2: Data Restoration (New Computer)

The client installs the software on their new PC and activates the license. To recover billing records:

$0

$0

$0

$0

$0

$0

$0

$0

Encryption of Backups

Backups must be encrypted so they cannot be read outside the application.

$0

$0

Backup Recovery Script

```
javascript
// backend/src/services/backup.service.js
const fs = require('fs');
const crypto = require('crypto');
const path = require('path');

const BACKUP_DIR = path.join(process.env.USERPROFILE, 'Documents', 'IthuNammaKada

// Encrypt backup file
function createBackup(dbFilePath, licenseKey) {
    if (!fs.existsSync(BACKUP_DIR)) fs.mkdirSync(BACKUP_DIR, { recursive: true })

    const timestamp = new Date().toISOString().replace(/[:.]/g, '-');
    const destPath = path.join(BACKUP_DIR, `backup-${timestamp}.bak`);

    const dbBuffer = fs.readFileSync(dbFilePath);

    // Derive Key using License key as secret
    const salt = crypto.randomBytes(16);
    const key = crypto.pbkdf2Sync(licenseKey, salt, 100000, 32, 'sha256');
    const iv = crypto.randomBytes(12);

    const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
    const encrypted = Buffer.concat([cipher.update(dbBuffer), cipher.final()]);
    const tag = cipher.getAuthTag();

    // Structure of file: [Salt (16B)][IV (12B)][Tag (16B)][Encrypted Data]
    const backupFileContent = Buffer.concat([salt, iv, tag, encrypted]);

    fs.writeFileSync(destPath, backupFileContent);
    return destPath;
}

// Restore backup file
function restoreBackup(backupFilePath, dbFilePath, licenseKey) {
    const backupContent = fs.readFileSync(backupFilePath);

    const salt = backupContent.subarray(0, 16);
    const iv = backupContent.subarray(16, 28);
    const tag = backupContent.subarray(28, 44);
    const encryptedData = backupContent.subarray(44);
```

```
    const key = crypto.pbkdf2Sync(licenseKey, salt, 100000, 32, 'sha256');

    const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
    decipher.setAuthTag(tag);

    const decrypted = Buffer.concat([decipher.update(encryptedData), decipher.fin

    // Overwrite existing DB file
    fs.writeFileSync(dbFilePath, decrypted);
}
```

$0

14. Hardware Printing Control (TSC TE244 & ESC/POS)

Integration of hardware features uses a Gatekeeper pattern, where raw interface access is wrapper-blocked unless the local license verification allows it.

TSC TE244 Barcode Printing using TSPL

We interface using COM/USB ports via raw Node.js byte writers (e.g. `node-printer` or raw `serialport`).

```javascript
// backend/src/services/barcode.service.js
const printer = require('pdf-to-printer'); // or raw USB raw writer
const { checkLicenseStatus } = require('./license.service');

async function printBarcodeLabel(product) {
    // 1. License Check Gatekeeper
    const license = await checkLicenseStatus();
    if (!license.valid || !license.features.barcode_printing) {
        throw new Error('Barcode Printing feature is disabled in your current lic
    }

    // 2. Generate TSPL Command payload for TSC TE244
    // Command Reference:
    // SIZE Width, Height (in mm)
    // GAP distance (in mm)
    // DIRECTION 1 (Default orientation)
    // CLS (Clear buffer)
    // BARCODE X, Y, CodeType, Height, HumanReadable, Rotation, NarrowRatio, Wide
    // PRINT Quantity
    const tsplCommands = [
        'SIZE 50 mm, 25 mm',
        'GAP 3 mm, 0',
        'DIRECTION 1',
        'CLS',
        `TEXT 40,30,"ROMAN.TTF",0,1,1,"${product.name.substring(0, 15)}"`,
        `TEXT 40,60,"ROMAN.TTF",0,1,1,"MRP: Rs. ${product.mrp}"`,
        `BARCODE 40,90,"128",80,1,0,2,2,"${product.barcode}"`,
        'PRINT 1,1',
        '\r\n'
    ].join('\n');

    // 3. Write raw commands directly to TSC printer spooler
    const fs = require('fs');
    // On Windows, raw write to port can be sent directly to mapped printer port:
    fs.writeFileSync('\\\\localhost\\TSC_TE244', tsplCommands);
}
```

Thermal Receipt Printing using ESC/POS

```
javascript
// backend/src/services/thermal.service.js
const fs = require('fs');
const { checkLicenseStatus } = require('./license.service');

async function printThermalReceipt(billData) {
    const license = await checkLicenseStatus();
    if (!license.valid || !license.features.thermal_printing) {
        throw new Error('Thermal receipt printing is disabled under your license.
    }

    // ESC/POS Commands:
    // ESC @ (Initialize Printer) -> \x1b\x40
    // ESC a 1 (Align Center) -> \x1b\x61\x01
    // GS ! 1 (Double height text) -> \x1d\x21\x10
    const ESC = '\x1b';
    const GS = '\x1d';

    let escPosBytes = '';
    escPosBytes += ESC + '@'; // Initialize
    escPosBytes += ESC + 'a' + '\x01'; // Center Align
    escPosBytes += GS + '!' + '\x11'; // Double Size Title
    escPosBytes += `${license.shopName}\n`;
    escPosBytes += GS + '!' + '\x00'; // Normal Font
    escPosBytes += `Mob: ${license.ownerMobile}\n`;
    escPosBytes += '--------------------------------\n';
    escPosBytes += ESC + 'a' + '\x00'; // Left Align

    billData.items.forEach(item => {
        const itemLine = `${item.name.padEnd(18)} x${item.qty.toString().padEnd(3
        escPosBytes += itemLine;
    });

    escPosBytes += '--------------------------------\n';
    escPosBytes += `NET AMOUNT: Rs. ${billData.netAmount.toFixed(2).padStart(15)}
    escPosBytes += ESC + 'a' + '\x01'; // Center Align
    escPosBytes += 'Thank you! Visit again!\n\n\n\n';
    escPosBytes += ESC + 'i'; // Cut Paper Command (ESC i)

    // Spool command out to receipt printer port
```

```
    fs.writeFileSync('\\\\localhost\\ThermalPrinter', escPosBytes, 'binary');
}
```

$0

15. Best Practices & Production Guidelines

1. Offline-First Architecture Pattern

Since the application must operate completely offline, **never make blocking network calls during POS billing transitions**.

$0

$0

$0

2. Auto-Updates and Version Control

Use Electron's `electron-updater` package to pull code updates automatically when online:

$0

$0

$0

3. Application Execution Safeguards

$0

$0

Walkthrough & Verification Results

Licensing System Walkthrough & Deployment Guide

We have successfully built and verified the **central licensing backend** (`licensing-server`) and the **web management panel** (`admin-portal`).

Below is the verification of the working system, followed by the deployment guide explaining how you keep the licensing server under your control and convert your desktop app to be completely offline-first.

$0

1. Verified Admin Web Portal

The browser subagent logged into the admin dashboard on port 5180 and successfully created a **3-Month (Quarterly) Subscription** for *"Siva Stores"*.

Here is the screenshot of the live admin dashboard, displaying the newly generated license key (`INK-3-M-...`) and customer details:

!Admin Portal Dashboard

$0

2. How to Host & Keep the Licensing Server on Your Side

The licensing server and admin portal are designed to run in the cloud, completely under your control.

Step 1: Host the Backend (`licensing-server`)

Deploy the Node.js/Express server to a cloud platform such as **Render**, **Railway**, **AWS**, or **DigitalOcean**. 1. Set up a free or cheap **MongoDB Atlas** database cluster. 2. In your cloud provider dashboard, configure the Environment Variables:

$0

$0

$0

3. The server will run at a URL like `https://licensing.ithunammakada.in`.

Step 2: Host the Admin Portal (`admin-portal`)

The admin portal is a React frontend that communicates with the `licensing-server`. 1. In `admin-portal/src/App.tsx`, change `const API_BASE = 'http://localhost:5500/api'` to your live server URL (e.g. `const API_BASE = 'https://licensing.ithunammakada.in/api'`). 2. Run `npm run build` in the `admin-portal` folder. This generates static HTML/JS/CSS assets in the `dist/` directory. 3. Upload the `dist/` folder to a hosting provider like **Netlify**, **Vercel**, or **GitHub Pages**. 4. Access the dashboard from your secure web URL. Only you can log in using your admin credentials.

$0

3. How to Convert the Desktop App to Run Offline-First

To convert your desktop billing software to use this licensing system, you integrate the **Public Ed25519 Key Check** into your Electron app.

```
mermaid
graph TD
    A[Customer Launches App] --> B{Does license.lic exist?}
    B -->|Yes| C[Read File & Generate Local Machine ID]
    B -->|No| G[Show React Activation Screen]
    C --> D{Verify Cryptographic Signature using Public Key}
    D -->|Invalid| G
    D -->|Valid| E{Is Machine ID matching local PC?}
    E -->|No| G
    E -->|Yes| F{Is Current Date < Expiry Date?}
    F -->|No| H[Show Expiry Lock Screen]
    F -->|Yes| I[Run SQLite / SQLCipher & Boot Billing App]
```

Electron Integration Script (`electron/main.js`)

Copy this code block into your Electron main process file:

```javascript
const { app, BrowserWindow, ipcMain } = require('electron');
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
const { generateStableMachineId } = require('./machineId'); // From Section 3 of

// Bundled public key used to verify signatures locally offline
const PUBLIC_KEY_PEM = `-----BEGIN PUBLIC KEY-----
MCowBQYDK2VwAyEAi43tLw7fN5c+rR7N97V4a9d701a2c3d4e5f6g7h8i9j=
-----END PUBLIC KEY-----`;

const LICENSE_PATH = 'C:\\ProgramData\\IthuNammaKada\\license.lic';

// Verify local license file offline
function checkLicenseOffline() {
    try {
        if (!fs.existsSync(LICENSE_PATH)) {
            return { valid: false, reason: 'NO_LICENSE' };
        }

        const rawContent = fs.readFileSync(LICENSE_PATH, 'utf8');
        const { data, signature } = JSON.parse(rawContent);

        // 1. Verify cryptographic signature using the public key
        const isSignatureValid = crypto.verify(
            null,
            Buffer.from(JSON.stringify(data)),
            PUBLIC_KEY_PEM,
            Buffer.from(signature, 'hex')
        );

        if (!isSignatureValid) {
            return { valid: false, reason: 'TAMPERED' };
        }

        // 2. Verify Machine ID binding
        const currentMachineId = generateStableMachineId();
        if (data.machineId !== currentMachineId) {
            return { valid: false, reason: 'INVALID_MACHINE' };
        }
```

```
        // 3. Verify expiration date
        if (data.expiresAt && Date.now() > new Date(data.expiresAt).getTime()) {
            return { valid: false, reason: 'EXPIRED', data };
        }

        return { valid: true, data };
    } catch (err) {
        return { valid: false, reason: 'ERROR', message: err.message };
    }
}

// IPC Handlers for React Frontend communication
ipcMain.on('check-license-status', (event) => {
    const status = checkLicenseOffline();
    event.reply('license-status-response', status);
});

// Saves the signed license file from the activation screen
ipcMain.on('save-license-file', (event, signedLicenseObject) => {
    try {
        const dir = path.dirname(LICENSE_PATH);
        if (!fs.existsSync(dir)) fs.mkdirSync(dir, { recursive: true });

        fs.writeFileSync(LICENSE_PATH, JSON.stringify(signedLicenseObject, null,
        event.reply('save-license-response', { success: true });
    } catch (err) {
        event.reply('save-license-response', { success: false, error: err.message
    }
});
```

$0

4. How the Customer Activates Offline

If a customer does not have internet access on their shop PC, they use the **Offline Activation Workflow**:

1. **Get Machine ID**: On first launch, the desktop app notices no license file is found. It redirects the customer to an activation screen showing their generated **Machine ID**: `D59B25A7...`. 2. **Contact Support**: The customer takes a photo or copies their **Machine ID** and **License Key**, and sends them to you via WhatsApp. 3. **Generate File**: You log in to your **Admin Portal**, click the download icon next to the user's License Key,

paste their Machine ID, and click **Download .lic File**. 4. **Deliver File**: You send the generated `ithu_namma_kada.lic` file back to the customer on WhatsApp. 5. **Import & Run**: The customer copies the file to a USB drive, plugs it into their shop PC, clicks **Import License** in the desktop app, and uploads the file. The app validates the signature, writes it to `%PROGRAMDATA%`, and immediately unlocks the POS for offline operation.